

Deep Neural Networks

Lab 1 – Environment Setup & Your First Neural Network

Module: CMP6232 Deep Neural Networks and Ethics

Session length: 3 hours

Introduction

Welcome to the first laboratory session of CMP6232: Deep Neural Networks and Ethics. Throughout this module we will use TensorFlow 2.x with Keras (which is integrated within TensorFlow) to build deep learning models. The recommended Python version is 3.10 or later. Google Colab is our primary development environment, but you may also set up a local environment using Anaconda.

In this first session we will install and familiarise ourselves with these tools, then run a small neural network to confirm everything works.

In this lab you will

- Set up a working deep learning environment (Colab or Anaconda).
- Run and verify your first Keras neural network on the MNIST dataset.
- Explore the dataset the model learns from.
- Improve the model by adding a hidden layer and compare the results.

1. Setting Up Your Environment

You have two options. If you are unsure, use Option 2 (Google Colab); it needs no installation.

Option 1 – Local installation using Anaconda

If you prefer to work locally on your own machine, download and install Anaconda (which includes the Spyder IDE) from anaconda.com/download. Installing Anaconda automatically provides many scientific-computing libraries such as NumPy, SciPy, scikit-learn, h5py and Pillow.

A few packages used in later sessions must be installed by you, for example TensorFlow and Keras. Open the Anaconda Prompt (Anaconda3) and run:

```
conda install tensorflow
conda install keras
```

Note Keras ships inside TensorFlow 2.x, so installing TensorFlow is usually enough. You can also install the packages through the Anaconda Navigator GUI (guide: freecodecamp.org/news/install-tensorflow-and-keras-using-anaconda-navigator-without-command-line/).

Once the packages are installed, open Spyder and start writing your code.

```

Anaconda Prompt (Anaconda3): conda install keras

(base) C:\Users\moham\Anaconda3:
Collecting package metadata (current_repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: C:\Users\moham\Anaconda3
  added / updated specs:
    - keras

The following packages will be downloaded:

package | build | size
-----|-----|-----
ca-certificates-2020.7.22 | 0 | 125 KB
conda-4.8.4 | py37_0 | 2.9 MB
Total: 3.0 MB

The following packages will be UPDATED:

ca-certificates | 2020.6.24_0 -> 2020.7.22_0
conda | 4.8.3-py37_0 -> 4.8.4-py37_0

Proceed ([y]/n)?

```

Figure 1. Installing TensorFlow and Keras from the Anaconda Prompt.

Option 2 – Google Colab (recommended)

Google Colab is a free, cloud-based notebook environment. You do not need to install anything, and it provides a free GPU. Follow the steps below.

1. Sign in with your Google account and open Google Drive.

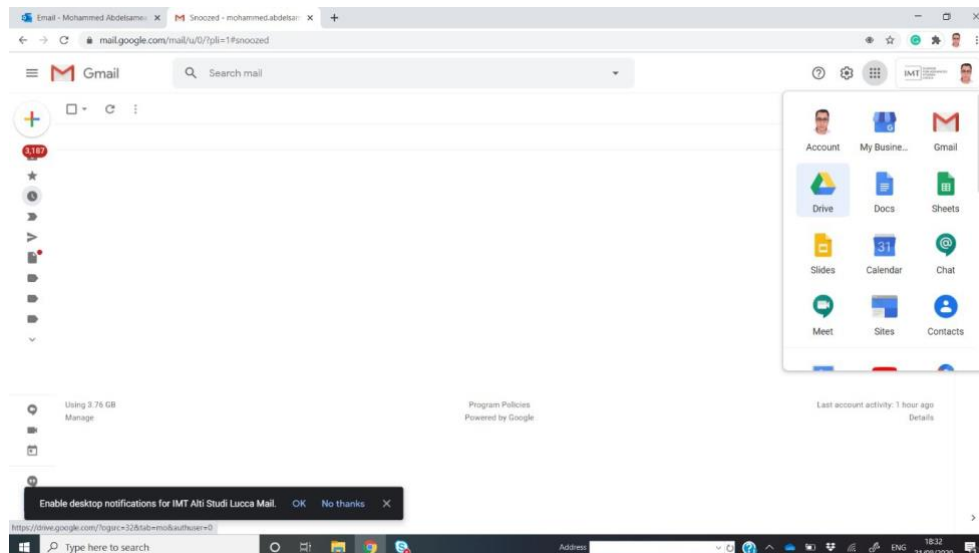


Figure 2. Accessing Google Drive.

2. In Drive, choose New → More → Google Colaboratory.

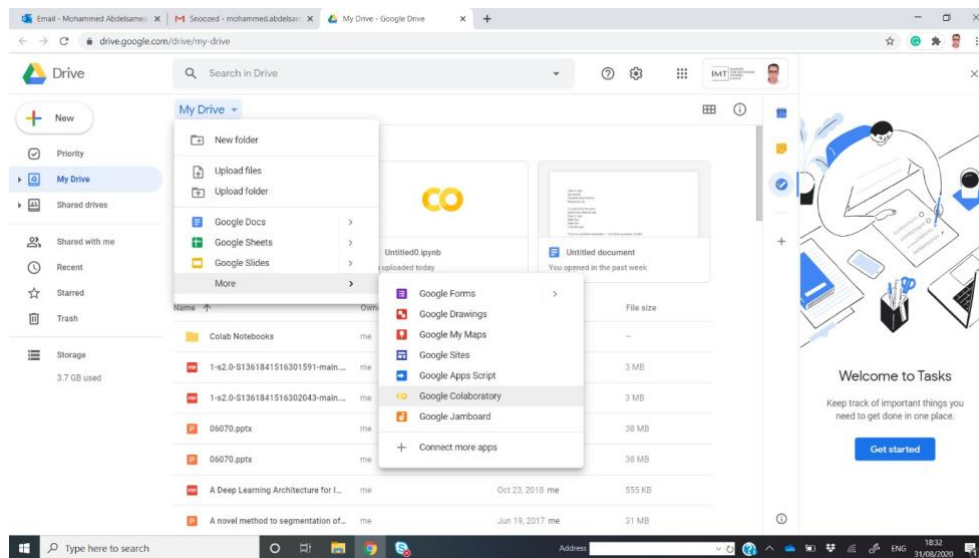


Figure 3. Creating a new Colab notebook.

3. A new notebook opens. It works like a Jupyter notebook.

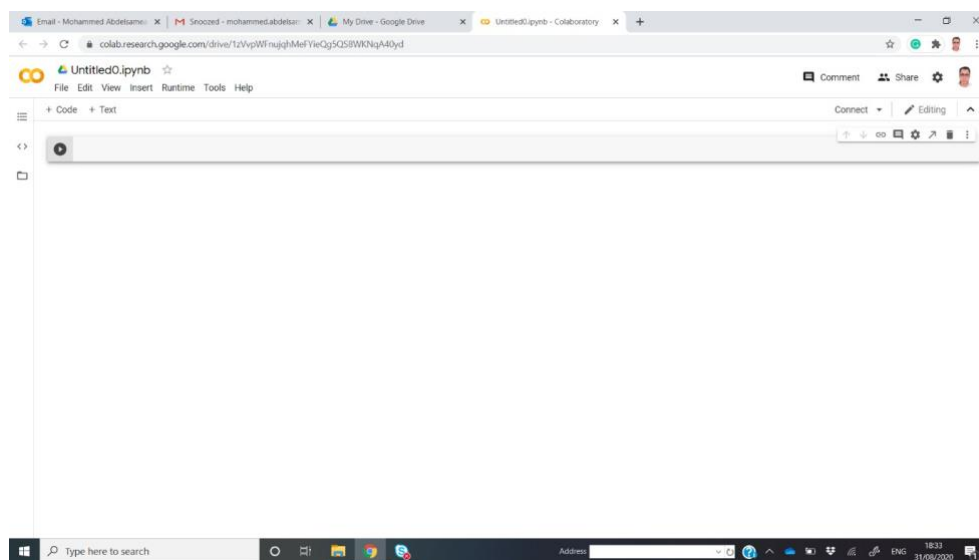


Figure 4. The Colab notebook interface.

4. Rename your notebook by clicking its title at the top.

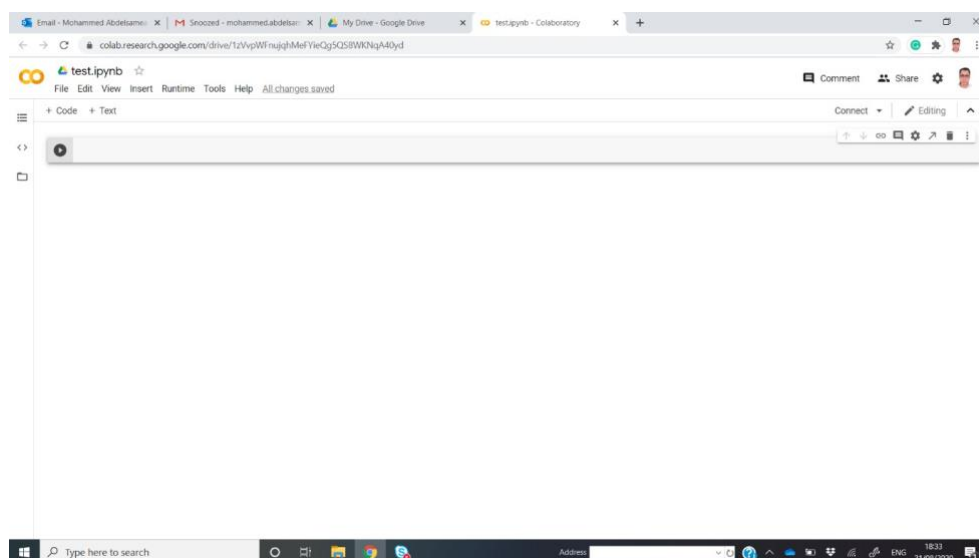


Figure 5. Renaming the notebook.

5. Add explanatory notes with '+ Text'.

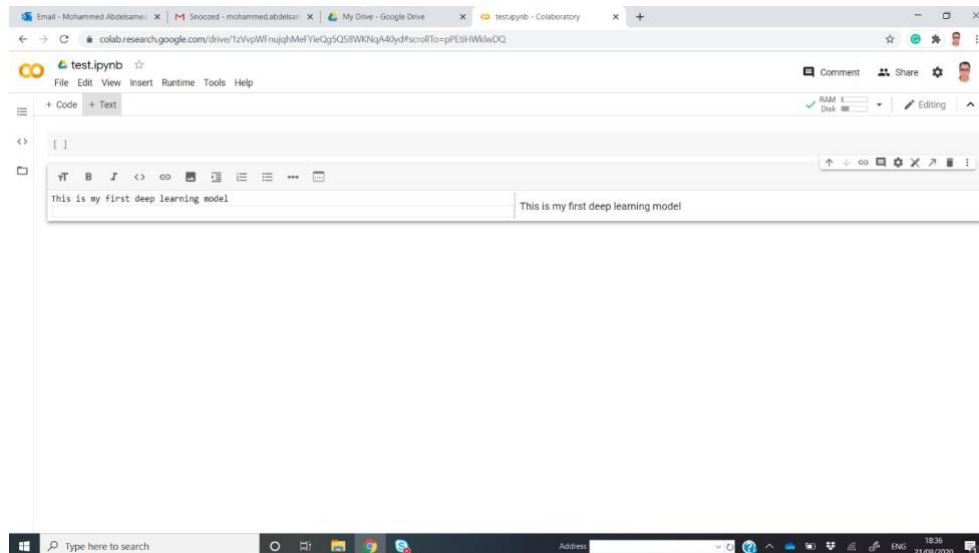


Figure 6. Adding a text cell.

6. Add Python code with '+ Code'.

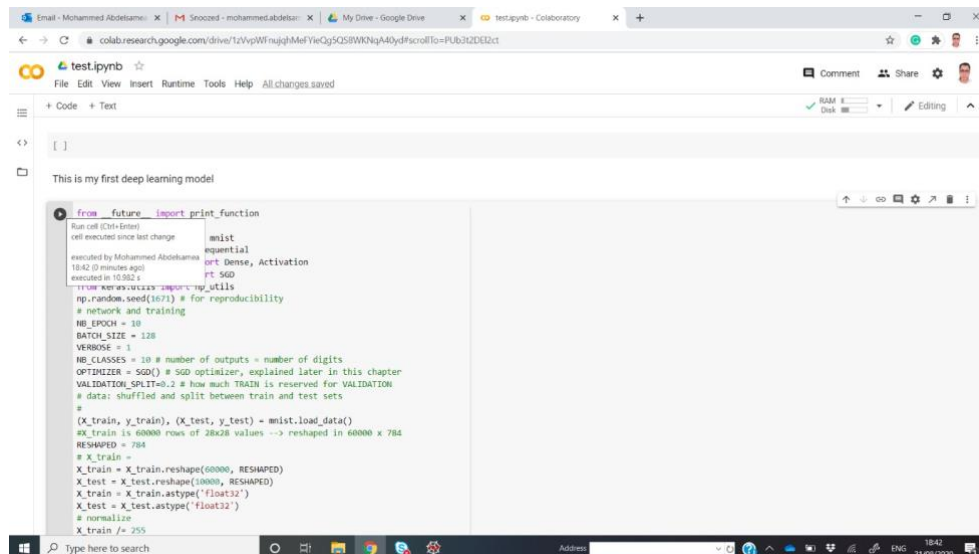


Figure 7. Adding a code cell.

7. Run a cell by pressing the play button (or Shift + Enter).

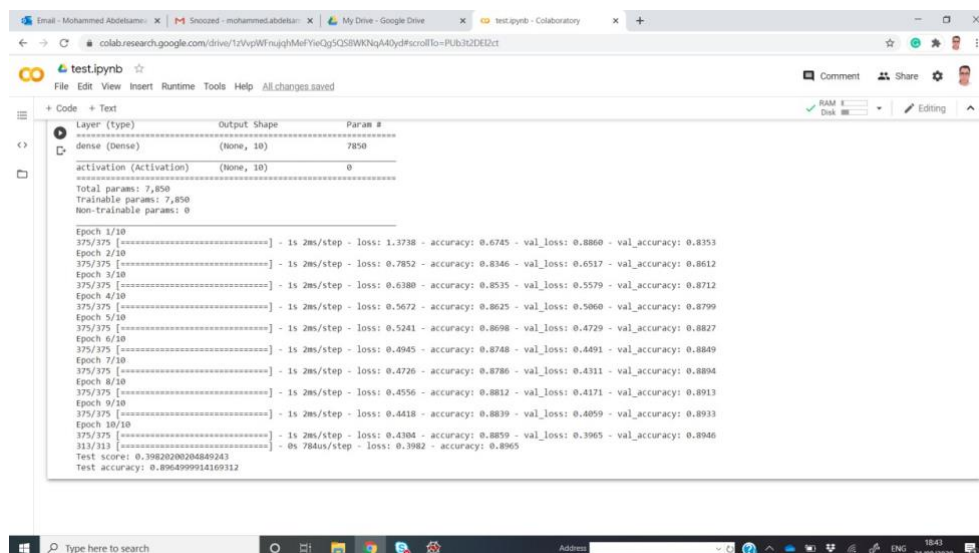


Figure 8. Running a cell.

Tip Save your notebook (File → Save a copy in Drive) so you keep your work from every lab in one place.

2. Activity 1 – Verify Your Setup

Copy the code below into a Colab code cell (or a Spyder script) and run it. It trains a simple neural network on the MNIST dataset of handwritten digits and prints the test accuracy. If it runs without errors, your environment is ready.

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.utils import to_categorical
import numpy as np

# Set random seed for reproducibility
np.random.seed(1671)

# Hyperparameters
NB_EPOCH = 20          # number of training epochs (increase to train longer)
BATCH_SIZE = 128       # samples per gradient update
VERBOSE = 1           # 1 = progress bar, 0 = silent, 2 = one line per epoch
NB_CLASSES = 10        # number of output classes (digits 0-9)
OPTIMIZER = SGD()      # Stochastic Gradient Descent optimiser
VALIDATION_SPLIT = 0.2 # fraction of training data used for validation
RESHAPED = 784         # 28 x 28 pixels = 784 features per sample

# Load and prepare the MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train.reshape(60000, RESHAPED).astype('float32') / 255
X_test = X_test.reshape(10000, RESHAPED).astype('float32') / 255
print(X_train.shape[0], 'train samples')
print(X_test.shape[0], 'test samples')

# Convert class labels to one-hot encoded vectors
Y_train = to_categorical(y_train, NB_CLASSES)
Y_test = to_categorical(y_test, NB_CLASSES)

# Build the neural network model
model = Sequential()
model.add(Dense(NB_CLASSES, input_shape=(RESHAPED,)))
model.add(Activation('softmax')) # softmax for multi-class classification
model.summary()

# Compile and train
model.compile(loss='categorical_crossentropy', optimizer=OPTIMIZER, metrics=['accuracy'])
history = model.fit(X_train, Y_train,
                    batch_size=BATCH_SIZE, epochs=NB_EPOCH,
                    verbose=VERBOSE, validation_split=VALIDATION_SPLIT)

# Evaluate on the test set
score = model.evaluate(X_test, Y_test, verbose=VERBOSE)
print('Test score:', score[0]) # loss on the test set
print('Test accuracy:', score[1]) # accuracy on the test set
```

What this code does

- **Loads MNIST** – 70,000 greyscale images of handwritten digits (28 × 28 pixels).
- **Prepares the data** – flattens each image to a 784-value vector and scales pixels to the range 0–1.
- **Builds a model** – a single dense layer with a softmax output that predicts one of ten digit classes.
- **Trains and evaluates** – fits the model, then reports the loss and accuracy on unseen test data.

Expected result This simple model should reach roughly 90–92% test accuracy. Note the accuracy; we will try to beat it in Activity 3.

3. Activity 2 – Explore the Data

A model can only learn from the data it is given, so it is worth looking at that data first. Run the code below in a new cell to inspect the MNIST images and their labels.

```
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras.datasets import mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()
print('Training images:', x_train.shape)  # (60000, 28, 28)
print('Test images:', x_test.shape)      # (10000, 28, 28)
print('Classes:', np.unique(y_train))    # digits 0-9

# Show the first 10 training images with their labels
plt.figure(figsize=(10, 2))
for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(x_train[i], cmap='gray')
    plt.title(str(y_train[i]))
    plt.axis('off')
plt.show()
```

Try this

- How many training and test images are there? What is the shape of a single image?
- Change the range in the loop to view different digits.
- Add `print(x_train[0])` to see the raw pixel values behind one image.

4. Activity 3 – Improve the Model

The model in Activity 1 has no hidden layer. Adding one lets the network learn richer patterns. Run this in a new cell, after Activity 1 (it reuses the same prepared data).

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam

# Reuses X_train, Y_train, X_test, Y_test, RESHAPED, NB_CLASSES,
# BATCH_SIZE, NB_EPOCH and VALIDATION_SPLIT from Activity 1.
N_HIDDEN = 128

model = Sequential()
model.add(Dense(N_HIDDEN, input_shape=(RESHAPED,), activation='relu')) # hidden layer
model.add(Dropout(0.3)) # reduces overfitting
model.add(Dense(NB_CLASSES, activation='softmax')) # output layer
model.summary()

model.compile(loss='categorical_crossentropy', optimizer=Adam(), metrics=['accuracy'])
history = model.fit(X_train, Y_train,
                    batch_size=BATCH_SIZE, epochs=NB_EPOCH,
                    verbose=VERBOSE, validation_split=VALIDATION_SPLIT)

score = model.evaluate(X_test, Y_test, verbose=VERBOSE)
print('Test accuracy:', score[1])
```

Try this

- Compare the test accuracy with Activity 1. How much did it improve?
- Change N_HIDDEN (for example 64 or 256) and re-run. What happens?
- Swap the optimiser back to SGD() and compare it with Adam().

Expected result With one hidden layer and the Adam optimiser, test accuracy usually rises to around 97–98%.

Before You Leave

- Your environment runs the Activity 1 code without errors.
- You have recorded the test accuracy from Activities 1 and 3.
- You have saved your notebook to Google Drive (File → Save a copy in Drive).